

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

- ABINAYA.R (E25AI102)

Scenario

The Medical shop company wants to maintain information about different products and their categories. Each product belongs to a particular category. The system should store and manage details such as **Product Name, Price, and Stock Availability**.

1. Design Product and Category Tables

Category Table

The Category table is used to store information about different product categories. The **Category_ID** uniquely identifies each category.

```
SQL> CREATE TABLE Category1(Category_ID VARCHAR2(7),Category_Name VARCHAR2(20),Description VARCHAR2(50));
Table created.
```

Product Table

The Product table is used to store details about the products. It contains information such as **Product_ID, Product_Name, Price, Stock, and Category_ID**.

```
SQL> CREATE TABLE Product1(Product_ID VARCHAR2(7),Product_Name VARCHAR2(20),Category_ID VARCHAR2(7),Price NUMBER(9,2),Stock_Quantity Number);
Table created.
```

The two tables, **Category** and **Product**, have been created. Now, we need to define the relationship between the tables and apply the required constraints.

2. Define Relationships and Constraints

Relationship Between Category and Product

A category can contain many products, but each product belongs to only one category.

Therefore, the relationship between the two tables is:

One Category → Many Products

**Relationship: One-to-Many (1 :)*

Primary Key

A **Primary Key** is a column that uniquely identifies each record in a table. It does not allow duplicate values.

- **Category_ID** is the Primary Key in the Category table.

- **Product_ID** is the Primary Key in the Product table.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_category PRIMARY KEY(category_ID);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT PK_product PRIMARY KEY(product_ID);  
Table altered.
```

Foreign Key

A **Foreign Key** is used to connect two tables.

The **Category_ID** in the Product table acts as the Foreign Key. It refers to the **Category_ID** in the Category table.

```
SQL> ALTER TABLE product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(category_ID);  
Table altered.
```

NOT NULL Constraint

The **NOT NULL** constraint is used when a column must contain a value. It does not allow the field to be left empty.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(100) NOT NULL;  
Table altered.
```

CHECK Constraint

The **CHECK** constraint is used to make sure that the value entered satisfies a particular condition.

For example, the product price can be checked to make sure that it is greater than zero.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (PRICE > 0);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE Constraint

The **UNIQUE** constraint is used to make sure that duplicate values are not entered into a column.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);  
Table altered.
```

3. Insert Values

Category Table

Values are inserted into the Category table using the **INSERT** command.

```
SQL> INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description');  
Enter value for category_id: CAT101  
Enter value for category_name: Medicine  
Enter value for description: General medicines  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT101','Medicine','General medicines')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: CAT102  
Enter value for category_name: Pain relief  
Enter value for description: Pain relief tablets and medicines  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT102','Pain relief','Pain relief tablets and medicines')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: CAT103  
Enter value for category_name: Antibiotics  
Enter value for description: Antibiotic medicines  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT103','Antibiotics','Antibiotic medicines')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: CAT104  
Enter value for category_name: Vitamins  
Enter value for description: Vitamins Supplements  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT104','Vitamins','Vitamins Supplements')  
  
1 row created.
```

```
SQL> /  
Enter value for category_id: CAT105  
Enter value for category_name: First Aid  
Enter value for description: First id products  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT105','First Aid','First id products')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: CAT106  
Enter value for category_name: Skin care  
Enter value for description: Women and men skin care  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT106','Skin care','Women and men skin care')  
  
1 row created.  
  
SQL> /  
Enter value for category_id: CAT107  
Enter value for category_name: Baby care  
Enter value for description: New born baby care products  
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')  
new 1: INSERT INTO Category VALUES('CAT107','Baby care','New born baby care products')  
  
1 row created.
```

Product Table

Values are inserted into the Product table using the **INSERT** command.

```
SQL> INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity);
Enter value for product_id: p101
Enter value for product_name: paracetamol
Enter value for category_id: CAT101
Enter value for price: 25
Enter value for stock_quantity: 100
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('p101','paracetamol','CAT101',25,100)

1 row created.

SQL> /
Enter value for product_id: P102
Enter value for product_name: Ibuprofen
Enter value for category_id: CAT102
Enter value for price: 40
Enter value for stock_quantity: 80
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P102','Ibuprofen','CAT102',40,80)

1 row created.

SQL> /
Enter value for product_id: P103
Enter value for product_name: Amoxilin
Enter value for category_id: CAT103
Enter value for price: 75
Enter value for stock_quantity: 60
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P103','Amoxilin','CAT103',75,60)

1 row created.

SQL> /
Enter value for product_id: P104
Enter value for product_name: Vitamin C
Enter value for category_id: CAT104
Enter value for price: 120
Enter value for stock_quantity: 50
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
```

```
SQL> /
Enter value for product_id: P105
Enter value for product_name: Bandage
Enter value for category_id: CAT105
Enter value for price: 30
Enter value for stock_quantity: 100
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P105','Bandage','CAT105',30,100)

1 row created.

SQL> /
Enter value for product_id: P106
Enter value for product_name: Face cream
Enter value for category_id: CAT106
Enter value for price: 150
Enter value for stock_quantity: 20
old 1: INSERT INTO Product VALUES('&Product_ID','&Product_Name','&Category_ID',&Price,&Stock_Quantity)
new 1: INSERT INTO Product VALUES('P106','Face cream','CAT106',150,20)

1 row created.
```

Like this, insert a few more rows.

CRUD Operations

CRUD stands for:

- **C – Create**
-

- **R – Read**
- **U – Update**
- **D – Delete**

CRUD operations are used to manage the records stored in the database.

Insert Record

A new product can be added to the Product table using INSERT command

```
SQL> INSERT INTO Product(Product_ID, Product_Name, Category_ID, Price, Stock_Quantity)
2  VALUES('P101', 'Cough Syrup', 'CAT101', 190.00, 70);

1 row created.
```

Update Price

```
SQL> UPDATE Product
2  SET Price = 200.00,
3      Stock_Quantity = 80
4  WHERE Product_ID = 'P101'

1 row updated.
```

The price of an existing product can be changed using the **UPDATE** command.

Delete a Product

An existing product can be removed from the Product table using the **DELETE** command.

```
SQL> DELETE FROM Product
2  WHERE Product_ID = 'P101';

1 row deleted.
```

4. Generate Category-wise Product Reports

1. Display all products under each category

This report is used to display all the products belonging to each category.

```
SQL> SELECT C.CATEGORY_ID,C.CATEGORY_NAME,P.PRODUCT_ID,P.PRICE,P.STOCK_QUANTITY FROM Category C JOIN Product P ON C.CATEGORY_ID=P.CATEGORY_ID ORDER BY C.CAT
EGORY_ID;
```

CATEGORY_NAME	PRODUCT	PRICE	STOCK_QUANTITY
Medicine	p101	25	100
Pain relief	P102	40	80
Antibiotics	P103	75	60
Vitamins	P104	120	50

5 rows selected.

2. Count the number of products in each category

This report shows how many products are available under each category.

```
SQL> SELECT C.CATEGORY_NAME,COUNT(P.PRODUCT_ID) AS PRODUCT_COUNT FROM Product P JOIN Category C ON P.CATEGORY_ID=C.CATEGORY_ID GROUP BY C.CATEGORY_NAME;
```

CATEGORY_NAME	PRODUCT_COUNT
Antibiotics	1
Vitamins	1
First Aid	1
Medicine	1
Skin care	1
Pain relief	1

6 rows selected.

3. Find the highest-priced product in each category

This report is used to find the product with the highest price in every category.

```
SQL> SELECT C.CATEGORY_NAME,P.PRODUCT_NAME,P.PRICE AS HIGHEST_PRICE FROM Product P JOIN Category C ON P.CATEGORY_ID=C.CATEGORY_ID WHERE P.PRICE=(SELECT MAX(P2.PRICE) FROM Product P2 WHERE P2.CATEGORY_ID=P.CATEGORY_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Medicine	paracetamol	25
Pain relief	Ibuprofen	40

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Antibiotics	Amoxilin	75
Vitamins	Vitamin C	

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
		120

4. Display categories having more than 5 products

This report displays only those categories that contain more than five products.

```
SQL> SELECT c.Category_Name FROM Product P JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_name HAVING COUNT(p.Product_ID) > 5;
no rows selected
```

5. Find the average product price category-wise

This report calculates the average price of products available in each category.

```
SQL> SELECT C.CATEGORY_NAME,AVG(P.PRICE) AS AVG_PRICE FROM Product P JOIN Category C ON P.CATEGORY_ID=C.CATEGORY_ID GROUP BY C.CATEGORY_NAME
```

CATEGORY_NAME	AVG_PRICE
Antibiotics	75
Vitamins	120
First Aid	30

CATEGORY_NAME	AVG_PRICE
Medicine	25
Skin care	150
Pain relief	40

6 rows selected.